

MVP/скелет системи бронювання та операційного управління для Eurotrips,

Що вже є:

- Prisma-модель великої операційної системи (Prisma — спеціальна бібліотека, яка дозволяє програмістам працювати з базою даних через зрозумілий код, не пишучи складні SQL-запити вручну. Вона зменшує кількість помилок і пришвидшує розробку)
- seed-дані,
- QA-тести,
- DevOps-інфраструктура,
- фронтенд-прототипи та окремі заготовки інтеграцій.
- робочий Fastify API (Fastify — це фреймворк для Node.js. Він приймає запити від сайту або мобільного застосунку та виконує потрібні дії: авторизацію, створення бронювання, розрахунок фінансів тощо. Fastify обрано через його високу швидкість роботи та невелике навантаження на сервер.)

Підключено в API зараз: Auth, Tours, Bookings, Finance, placeholder Agents.

Запроєктовано в схемі/документах: CRM-ліди, агенти, комісії, готелі, транспорт, документи, комунікації, аналітика, rooming/розсадка, LiqPay, email, Zoho.

1. Загальна ідея проєкту

Проєкт має стати єдиною системою для туроператора Eurotrips, яка веде повний цикл:

- каталог турів;
- бронювання B2C та B2B через агентів;
- облік туристів;
- агентські комісії;
- платежі та заборгованості;
- операційне ведення турів: готелі, транспорт, активності, rooming;
- документи, повідомлення, аудит;
- фінансову аналітику й P&L (звіт про прибутки та збитки)

2. Технічний стек

Бекенд:

- Node.js 20; (середовище виконання JavaScript на сервері)
Node.js виконує всю бізнес-логіку:
логін користувачів;
бронювання;
перевірку місць;
фінансові розрахунки;
взаємодію з базою даних;
інтеграції з платіжними системами.
- Fastify 4; (фреймворк для Node.js, який прискорює роботу на сервері, бо вже має готові стандартні рішення-приймає запит і викликає шаблонний код виконання запиту-вхід в систему, реєстрація і тд)
- TypeScript; (JavaScript із "перевіркою помилок")
- Prisma 5; (інструмент роботи з базою даних)
- PostgreSQL 16; (база даних)
- Redis 7; (надшвидке сховище даних у пам'яті сервера)
- JWT auth; (JWT (JSON Web Token) — цифровий пропуск)
- Zod для валідації; (перевірка правильності даних)
- Vitest для unit-тестів; (автоматична перевірка роботи системи)

- Swagger UI для API-документації.(API-документація, API (Application Programming Interface) — набір правил, за якими різні програми спілкуються між собою.)

Інфраструктура:

- Docker / docker-compose;
- Nginx dev/prod reverse proxy;
- pgAdmin, RedisInsight, MailHog у dev-профілі;
- backup/rollback/server setup scripts.

Фронтенд-прототип:

- React 18;
- TypeScript;
- Tailwind-like класи;
- Axios;
- Zustand-подібний auth store;
- lucide-react іконки;
- mock-дані.

3. Бекенд: що вже є

Точка входу:

src/main.ts

Збірка Fastify app:

src/app.ts

Глобально підключені компоненти

- Helmet; Допомагає захищати систему від поширених атак.
- CORS; Щоб сторонні сайти не могли використовувати API від імені користувачів
- rate limit; Обмеження кількості запитів.
- cookies; Невеликі файли, які браузер зберігає у себе.
- JWT; (хто звернувся;яку роль має;що йому дозволено)
- Redis plugin; Спосіб підключити Redis до Fastify
- Swagger UI на /documentation; технічна енциклопедія
- health-check /health; перевірка здоров'я системи
- API prefix /api/v1.

Підключені API-модулі:

- /api/v1/auth; (вхід; реєстрація; вихід; зміна пароля; перевірка користувача)
- /api/v1/tours; (тури)
- /api/v1/bookings; (бронювання)
- /api/v1/finance; (бухгалтерія)
- /api/v1/agents як placeholder, що повертає порожній список для дозволених ролей.

4. Auth-модуль

Файли:

src/modules/auth/auth.routes.ts

src/modules/auth/auth.service.ts

Функціонал:

- POST /auth/login — логін за email/password;
- POST /auth/register — реєстрація користувача;
- POST /auth/refresh — оновлення access token через refresh token у HttpOnly cookie;

- POST /auth/logout — logout, видалення refresh token з Redis;
- GET /auth/me — поточний користувач;
- POST /auth/change-password — зміна пароля й logout зі старих сесій.

Особливості:

- пароль хешується bcrypt;
- access token живе 15 хв;
- refresh token живе 30 днів;
- refresh token зберігається в Redis;
- після logout access token блокується через Redis blacklist;
- у JWT payload кладеться роль, agentId, agentType, networkId.

5. Ролі та доступи

RBAC guard:

src/shared/guards/rbac.guard.ts

Ролі в системі:

- admin;
- director;
- manager;
- ops;
- agent;
- accountant;
- tourist.

Основна логіка:

- фінанси доступні тільки внутрішнім ролям;
- агент бачить тільки свої бронювання;
- агент не має бачити собівартість туру;
- турист має бачити тільки свої бронювання;
- для агентів є захист від IDOR: не можна отримати чуже бронювання напряму по ID.

6. Tours-модуль

Файли:

src/modules/tours/tours.routes.ts

src/modules/tours/tours.service.ts

Реалізовані endpoints:

- GET /tours — список турів із фільтрами;
- GET /tours/:id — деталі туру;
- GET /tours/:id/availability — доступність місць;
- POST /tours — створення туру, тільки admin, ops;
- PUT /tours/:id — редагування туру, тільки admin, ops;
- PATCH /tours/:id/status — зміна статусу, admin, ops, director;
- DELETE /tours/:id/archive — soft archive, тільки admin.

Фільтри туру:

- статус;
- тип туру: bus/avia/combined;
- дата виїзду від/до;
- продукт;
- напрямок;

- місто виїзду;
- теги;
- тільки доступні;
- пагінація;
- сортування.

Бізнес-правила:

- costPrice не повертається агентам і туристам;
- завершений або скасований тур не можна редагувати;
- при зміні totalSeats перевіряється, щоб нова кількість не була меншою за вже заброньовані місця;
- статуси туру мають дозволені переходи;
- тур не можна перевести в open, якщо room structure у готельних бронюваннях не затверджена, крім isFastLaunch.

7. Bookings-модуль

Файл:

src/modules/bookings/bookings.routes.ts

Зараз це MVP, реалізовано тільки читання:

- GET /bookings — список бронювань;
- GET /bookings/:id — деталь бронювання.

Функціонал:

- пагінація;
- фільтр за статусом;
- фільтр за tourId;
- агент бачить тільки бронювання зі своїм agentId;
- менеджер/адмін бачить всі;
- деталь бронювання включає тур, контактного туриста, учасників, платежі;
- агент не може відкрити чуже бронювання;
- туристу закладено перевірку участі у бронюванні.

Що ще тільки запроєктовано, але не реалізовано як route:

- створення бронювання;
- зміна статусу;
- додавання платежу;
- cancel/refund flow;
- participants management;
- self-service вибір місця/типу номера.

8. Finance-модуль

Файл:

src/modules/finance/finance.routes.ts

Реалізовані endpoints:

- GET /finance/summary;
- GET /finance/debts;
- GET /finance/tours/:id/pnl.

Доступ:

- summary, debts: admin, director, manager, accountant;
- pnl: admin, director, accountant;

- агент отримує 403.

Функціонал:

- загальна кількість бронювань;
- кількість підтверджених;
- загальний дохід;
- зібраний дохід;
- список боргів за бронюваннями;
- P&L по туру: revenue, cost, gross profit, occupancy.

9. Prisma-схема майбутньої бази даних(БД)

Головне джерело моделі:

prisma/schema.prisma

У схемі 20 моделей:

- User — користувачі системи;
- Tourist — туристи/клієнти;
- AgentNetwork — агентські мережі;
- Agent — агенти;
- CancellationPolicy — політики скасування;
- Staff — гіді, турлідери, водії, координатори;
- Tour — каталог турів;
- Lead — CRM-ліди;
- Booking — бронювання;
- BookingTourist — учасники бронювання;
- Payment — платежі;
- AgentCommission — агентські комісії;
- Hotel — база готелів;
- HotelBooking — бронювання готелів по турах;
- TransportBooking — транспорт;
- TourActivity — екскурсії/активності;
- TourExtra — додаткові витрати;
- Document — документи;
- Communication — email/SMS/Telegram/Viber лог;
- AuditLog — журнал змін.

Ключові enum-и:

- ролі користувачів;
- типи агентів;
- статуси турів;
- статуси бронювань;
- статуси оплат;
- типи платежів;
- джерела лідів;
- статуси комісій;
- типи документів;
- канали комунікацій;
- типи номерів;
- статуси rooming: draft, approved, final.

10. Головні бізнес-правила

У CLAUDE.md описано набір критичних правил:

- місця в турі мають бронюватися тільки транзакційно;
- комісія агента рахується від базової ціни туру, а не від total amount;
- комісію можна виплачувати тільки після завершення туру;
- агент не бачить собівартість, маржу, net profit;
- є два типи агентів: standard і network;
- network agent має co-commission та royalty;
- статуси бронювання мають сувору state machine;
- скасування оператором має створювати refund;
- тур не відкривається без затвердженої структури номерів;
- rooming trigger має спрацьовувати при 20 підтверджених туристах або за 14 днів до виїзду;
- tourist self-service блокується після final rooming.

11. Seed-дані

Файл:

prisma/seed.ts

Seed створює:

- cancellation policies;
- staff/гідів;
- 5 турів;
- користувачів ролей admin, manager, ops, accountant, agent;
- agent network;
- standard і network агентів;
- туристів;
- 2 бронювання;
- agent commission;
- lead.

12. Фронтенд-прототип

Файли лежать тут:

database/frontend

Основні екрани/компоненти:

- App.tsx — маршрутизація;
- LoginForm.tsx — логін;
- ProtectedRoute.tsx — захист маршрутів;
- Tours.tsx — каталог турів;
- TourCard.tsx — картка туру;
- Bookings.tsx — список бронювань;
- BookingRow.tsx — рядок бронювання з деталями;
- AgentCabinet.tsx — кабінет агента;
- CommissionBadge.tsx — блок комісії;
- PaymentBlock.tsx — блок платежів;
- StatusBadge.tsx — статусні бейджі;
- statuses.ts — синхронізовані статуси;
- api.ts — Axios-клієнт із auto-refresh;
- authStore.ts, useAuth.ts, auth.ts — auth state/helpers.

Фронтенд підтримує сценарії:

- логін;
- redirect залежно від ролі;

- захист сторінок;
- каталог турів із пошуком, фільтрами, grid/list view;
- таблицю бронювань із пошуком, фільтрами, пагінацією;
- агентський кабінет: власні бронювання, комісія, доступні тури, royalty для network-агента.

Це **UI-прототип на моках**, а не повністю зібраний frontend app.

13. Інтеграції

Папка:

database/api integrations

Вже є заготовки для:

- LiqPay:
 - генерація checkout data/signature;
 - webhook /webhooks/liqpay;
 - оновлення платежів/статусів бронювання;
 - endpoint для створення платежу по бронюванню.
- Email через Brevo:
 - підтвердження бронювання;
 - нагадування про оплату;
 - pre-departure info;
 - підтвердження отримання платежу;
 - повідомлення про скасування;
 - повідомлення агенту.
- Zoho migration:
 - типи та міграційна логіка для перенесення/синхронізації даних.

Ці інтеграції **не підключені в основний src/app.ts**, тобто зараз вони існують як окремі заготовки/артефакти.

14. QA

Є Playwright-тести в:

- frontend/tests
- database/QA/tests

Покриті сценарії:

- успішний логін;
- неправильні credentials;
- валідація форми логіну;
- захист маршрутів;
- logout;
- RBAC для фінансів;
- агент не бачить costPrice;
- агент бачить тільки свої бронювання;
- IDOR-захист;
- 401 без токена.

Є unit-тести для AuthService.

15. DevOps

Основні файли:

- docker-compose.yml

- docker-compose.prod.yml
- Dockerfile
- nginx/nginx.dev.conf
- nginx/nginx.prod.conf
- scripts/backup.sh
- scripts/rollback.sh
- scripts/server-setup.sh

Підтримуються:

- локальний запуск PostgreSQL + Redis;
- dev tools: pgAdmin, RedisInsight, MailHog;
- production compose;
- Nginx reverse proxy;
- backup/rollback;
- server setup.

16. Поточний стан проекту

- **Архітектурно:** проєкт повної ERP/booking/ops система для туроператора.
- **База даних:** вже досить детальна
- **API:** MVP, реально працюють auth, tours, bookings read-only, finance read-only.
- **Фронтенд:** прототип кабінетів і сторінок, ще не інтегрований.
- **Інтеграції:** LiqPay, Brevo, Zoho підготовлені, але не підключені до основного app.
- **QA:** основа для auth/RBAC/e2e.
- **DevOps:** є docker/dev/prod база для запуску.